BCC362 · SISTEMAS DISTRIBUÍDOS

# Trabalho Prático 3

Replicação e Tolerância a Falhas em Sistemas Distribuídos

**Protocolo 1**

Primário-Backup

**Ricart-Agrawala**

Exclusão Mútua

**Tolerância a Falhas**

Opção 2 — Cluster Store


GRUPO

Camile Reis de Sousa

João Henrique Pedrosa de Souza

Theo dos Anjos Silva

CONTEXTO

# Do TP2 para o TP3

O que mudou e o que foi mantido

## TP2 — Base

- 5 clientes externos
- 5 nós no Cluster Sync
- Exclusão mútua: Ricart-Agrawala
- Recurso R: time.sleep() simulado
- Sem tolerância a falhas

→

## TP3 — Novo

- 5 clientes externos
- 5 nós no Cluster Sync ✓
- Ricart-Agrawala mantido ✓
- **Cluster Store com 3 nós reais**
- **Protocolo 1 (Primário-Backup)**
- **Tolerância a falhas — Opção 2**

ARQUITETURA

# Arquitetura do Sistema

CLIENTES (5)

Cliente A | Cliente B | Cliente C | Cliente D | Cliente E

UDP ↓

CLUSTER SYNC — Ricart-Agrawala (5 nós)

Node 1 | Node 2 | Node 3 | Node 4 | Node 5

TCP ↓ (quando na Seção Crítica)

CLUSTER STORE — Protocolo 1: Primário-Backup (3 nós)

Store 1 — PRIMÁRIO | Store 2 — backup | Store 3 — backup

← escrita replicada para backups →

PROTOCOLO 1

# Primário Fixo com Backups

Como as escritas são replicadas no Cluster Store

| | | |
|---|---|---|
| 1 | Sync Node | Entra na Seção Crítica via R.A. |
| 2 | Sync → Primário | WRITE {content, request_id} |
| 3 | Primário → Backups | WRITE_REPLICA para cada backup vivo |
| 4 | Backups → Primário | REPLICA_OK (confirma gravação) |
| 5 | Primário → Sync | WRITE_OK {replicas: N} |
| 6 | Sync Node | Libera a Seção Crítica |

## Regras do Protocolo

### Primário fixo

O nó com menor ID vivo é sempre o primário.

### Escrita síncrona

WRITE_OK só é enviado após todos os backups vivos confirmarem.

### Redirect

Backup que recebe WRITE direciona o Sync ao primário real.

### Idempotência

request_id único evita entradas duplicadas no log.

CLUSTER SYNC

# Exclusão Mútua: Ricart-Agrawala

Algoritmo mantido do TP2 — garantia de que apenas 1 nó acessa o Store por vez

### RELEASED

Nó não precisa do recurso. Responde OK imediatamente a qualquer pedido.

### WANTED

Nó quer entrar. Envia REQUEST com timestamp e aguarda N-1 OKs.

### HELD

Nó está na Seção Crítica. Bloqueia todos os outros (defer queue).

## Lógica de Decisão ao Receber REQUEST

| | |
|---|---|
| Estado = RELEASED | → OK imediato |
| Estado = HELD | → Defer (adia OK até sair da SC) |
| Estado = WANTED e meu TS < seu TS | → Defer (eu tenho prioridade) |
| Estado = WANTED e meu TS > seu TS | → OK imediato (ele pediu antes) |

FALHAS

# Tolerância a Falhas — Opção 2

Falhas por omissão ou queda em nós do Cluster Store

## 2.1 💀 Store falha sem pedido ativo

`store_node_down() + store_election_done()`

| DETECÇÃO | RESPOSTA | RESULTADO |
| --- | --- | --- |
| PING TCP a cada 5s detecta ausência de PONG | Eleição automática: menor ID vivo vira primário | Sistema continua operando com 2 nós |

## 2.2 ⏱️ Store falha com pedido pendente

`settimeout(8) + fallback ordered_store_ids`

| DETECÇÃO | RESPOSTA | RESULTADO |
| --- | --- | --- |
| Sync Node recebe timeout na conexão TCP | Retry automático: tenta próximo nó vivo em ordem | Escrita é completada no novo primário |

## 2.3 🔑 Store falha durante a escrita

`seen_requests set + store_duplicate_ignored()`

| DETECÇÃO | RESPOSTA | RESULTADO |
| --- | --- | --- |
| Sync retenta e pode atingir o nó recuperado | request_id único filtra duplicatas no log | Nenhuma entrada duplicada no audit log |

ELEIÇÃO

# Eleição do Novo Primário

Quando o primário falha, o backup com menor ID assume automaticamente

1. **PING periódico**
   Cada Store faz PING TCP nos outros a cada 5 segundos.
2. **Timeout sem PONG**
   Se não receber PONG em 3s, marca o nó como morto (alive_nodes).
3. **Eleição determinística**
   Todos usam a mesma regra: min(alive_nodes) = novo primário.
4. **Notificação PROMOTE**
   O nó que elegeu notifica os demais via mensagem PROMOTE.
5. **Sync atualiza cache**
   Na próxima escrita, recebe REDIRECT e atualiza store_primary_id.

✅ Garantia de consistência: todos os nós usam a mesma regra determinística (min ID), portanto chegam ao mesmo resultado sem coordenação adicional.

DEMO

# Observabilidade em Tempo Real

Logger estruturado + Dashboard web que consome o JSONL ao vivo

## logger.py

Emite cada evento em dois destinos:

- Terminal colorido com prefixo [TIME][TIPO][EVENTO]
- audit_structured.jsonl — uma linha JSON por evento
- Campos: ts, time, node_type, node_id, event, msg, data
- Helpers semânticos para cada tipo de evento do protocolo

## dashboard.html

Frontend HTML puro — abre direto no navegador:

- Polling a cada 1.5s no log_server (porta 8888)
- Painel com estado de cada Sync e Store Node
- Contadores ao vivo: writes, SC, falhas, eleições
- Timeline filtrável por tipo de evento

## Fluxo de Evento

store_node.py / node_cluster.py → logger.py → audit_structured.jsonl → log_server.py :8888 → dashboard.html

RESULTADOS

# Resultados Observados

## ✅ O que funcionou

- Ricart-Agrawala garantiu exclusão mútua: nenhuma colisão detectada no audit.log
- Protocolo 1: todas as escritas replicadas para 2 backups com REPLICA_OK
- Sub-caso 2.1: eleição automática em < 10s após docker pause store1
- Sub-caso 2.2: Sync Node retentou e completou escrita no novo primário
- Sub-caso 2.3: request_id impediu duplicatas mesmo com retry
- Dashboard exibiu mudança de estado do Store em tempo real

## ⚠️ Limitações conhecidas

### Janela de transição

Escrita pode falhar se o Sync tenta durante a eleição (store ainda não elegeu). Ocorreu 1 vez nos testes.

### Sem log de operações pendentes

Uma escrita na janela de falha é perdida. O Protocolo 3 (quórum) resolveria isso.

### Eleição não é atômica

Dois nós podem iniciar eleição simultaneamente. A regra determinística resolve, mas pode haver delay.

# Conclusão

O TP3 estendeu o TP2 com armazenamento distribuído real, replicação e tolerância a falhas por omissão e queda.

### Exclusão Mútua

Ricart-Agrawala distribuído garante que apenas um nó acessa o recurso por vez, sem coordenador central.

### Replicação

Protocolo 1 garante que toda escrita é confirmada em múltiplas cópias antes de responder ao cliente.

### Tolerância a Falhas

PING, eleição determinística e idempotência cobrem os 3 sub-casos de falha do Cluster Store.